

Advanced Classical ML and Variational Algorithms

QML

Contents

1	Overview	2
2	Support Vector Machines	2
2.1	Linear SVM: The Idea	2
2.2	Primal Formulation	2
2.3	Soft Margin SVM	3
2.4	Dual Formulation	3
3	The Kernel Trick	3
3.1	Feature Maps	3
3.2	The Kernel Function	3
3.3	Common Kernels	4
3.4	Mercer's Theorem	4
4	Neural Networks	5
4.1	The Perceptron	5
4.2	Multi-Layer Perceptron (MLP)	5
4.3	Activation Functions	5
4.4	Backpropagation	5
4.5	Universal Approximation Theorem	6
5	Connection Between Kernels and Neural Networks	6
5.1	Neural Tangent Kernel	6
5.2	The Two Paradigms of QML	7
6	Deep Learning Architectures (Brief)	7
6.1	Convolutional Neural Networks (CNNs)	7
6.2	Recurrent Neural Networks (RNNs)	7
6.3	Transformers and Attention	8
7	The Variational Principle	8
8	Summary	9

1 Overview

this lecture goes deep into the two classical ML paradigms that directly inspire QML: kernel methods and neural networks. understanding SVMs and the kernel trick is essential bcs quantum kernel methods r the most mathematically grounded approach to QML. and understanding neural networks is essential bcs variational quantum circuits (PQCs) r essentially the quantum analogue of neural networks.

by the end of this lecture, u should see exactly where quantum fits into the classical ML picture.

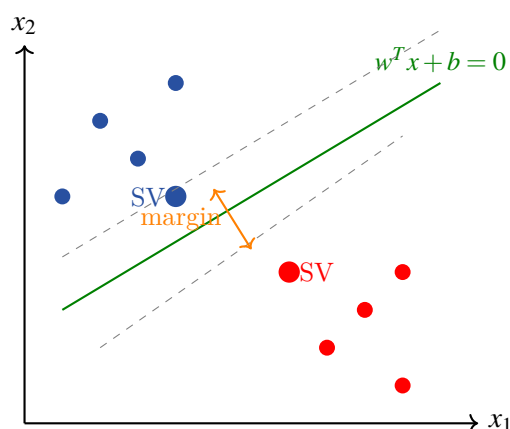
2 Support Vector Machines

2.1 Linear SVM: The Idea

Definition 2.1. a support vector machine (SVM) finds the hyperplane that separates two classes with the maximum margin — the maximum distance between the hyperplane and the nearest data points from each class.

the hyperplane is defined by: $w^T x + b = 0$

classification rule: $\text{sign}(w^T x + b)$



the points closest to the boundary r called support vectors — they “support” the hyperplane. only these points determine the decision boundary.

2.2 Primal Formulation

the optimization problem (hard margin SVM):

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1 \quad \forall i$$

the constraint says every point must be on the correct side of the margin. minimizing $\|w\|^2$ maximizes the margin (margin = $2/\|w\|$).

2.3 Soft Margin SVM

real data is rarely perfectly separable. the soft margin SVM allows some misclassifications by introducing slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

$C > 0$ is the regularization parameter:

- large C : penalize misclassifications heavily (closer to hard margin)
- small C : allow more misclassifications (smoother boundary, better generalization)

2.4 Dual Formulation

using lagrange multipliers $\alpha_i \geq 0$ for each constraint, the dual problem is:

$$\boxed{\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j (x_i^T x_j) \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C}$$

Key Point. the key observation: the dual formulation only depends on the data through inner products $x_i^T x_j$. the decision function is:

$$f(x) = \text{sign} \left(\sum_{i \in \text{SVs}} \alpha_i y_i (x_i^T x) + b \right)$$

this opens the door to the kernel trick: replace $x_i^T x_j$ with $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ for some feature map ϕ — without ever computing ϕ explicitly.

3 The Kernel Trick

3.1 Feature Maps

the problem with linear SVMs: many real-world datasets r not linearly separable. the solution: map the data to a higher-dimensional space where it is linearly separable.

Definition 3.1. a feature map $\phi : \mathcal{X} \rightarrow \mathcal{F}$ maps input data to a (possibly infinite-dimensional) feature space $\mathcal{F}$. the SVM is applied in $\mathcal{F}$ where the data may be linearly separable.

Example 3.1. data in $\mathbb{R}^2$: $x = (x_1, x_2)$.

feature map: $\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2) \in \mathbb{R}^3$.

a circle in the original space becomes a hyperplane in feature space.

3.2 The Kernel Function

Definition 3.2. a kernel function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ computes the inner product in feature space without explicitly computing the feature map:

$$\boxed{K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{F}}}$$

Key Point. the kernel trick: replace every inner product $\langle x_i, x_j \rangle$ in the SVM dual with $K(x_i, x_j)$. this implicitly works in the high-dimensional feature space $\mathcal{F}$ without ever computing $\phi(x)$ explicitly.

dual SVM with kernels:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

decision function:

$$f(x) = \text{sign} \left(\sum_{i \in \text{SVs}} \alpha_i y_i K(x_i, x) + b \right)$$

3.3 Common Kernels

linear kernel:

$$K(x, x') = x^T x'$$

feature space = input space. no nonlinearity.

polynomial kernel:

$$K(x, x') = (x^T x' + c)^d$$

feature space contains all monomials up to degree d .

Example 3.2. for $d = 2$, $c = 0$, $x \in \mathbb{R}^2$:

$$K(x, x') = (x_1 x'_1 + x_2 x'_2)^2 = x_1^2 x_1'^2 + 2x_1 x_2 x'_1 x'_2 + x_2^2 x_2'^2$$

this corresponds to $\phi(x) = (x_1^2, \sqrt{2}x_1 x_2, x_2^2)$ — the quadratic feature map.

radial basis function (RBF) / gaussian kernel:

$$K(x, x') = \exp \left(-\frac{\|x - x'\|^2}{2\sigma^2} \right)$$

Intuition. the RBF kernel is special bcs its feature space is infinite-dimensional. u can show that the corresponding feature map ϕ maps data into an infinite-dimensional hilbert space. the kernel trick lets u work in this infinite space implicitly — u only ever compute the kernel function, which is cheap.

this is directly analogous to quantum kernels: the quantum feature map $\phi(x) = U(x)|0\rangle$ maps data into the 2^n -dimensional hilbert space of n qubits. computing the kernel $K(x, x') = |\langle \phi(x') | \phi(x) \rangle|^2$ on a quantum computer is efficient, even tho the feature space is exponentially large.

3.4 Mercer's Theorem

Definition 3.3. Mercer's theorem: a function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a valid kernel (i.e., corresponds to some feature map and inner product) if and only if the kernel matrix (Gram matrix) $K_{ij} = K(x_i, x_j)$ is positive semidefinite for any set of data points $\{x_1, \dots, x_N\}$.

this is important: any positive semidefinite function can be used as a kernel. u dont need to find the feature map explicitly — u just need to verify the PSD property.

Key Point. quantum kernels r valid kernels bcs:

$$K(x, x') = |\langle \phi(x') | \phi(x) \rangle|^2 = \text{tr}(\rho(x)\rho(x'))$$

where $\rho(x) = |\phi(x)\rangle \langle \phi(x)|$. the kernel matrix $K_{ij} = \text{tr}(\rho(x_i)\rho(x_j))$ is always PSD bcs its a gram matrix of density operators under the hilbert-schmidt inner product.

4 Neural Networks

4.1 The Perceptron

Definition 4.1. a perceptron is the simplest neural network unit:

$$f(x) = \sigma(w^T x + b)$$

where σ is a nonlinear activation function.

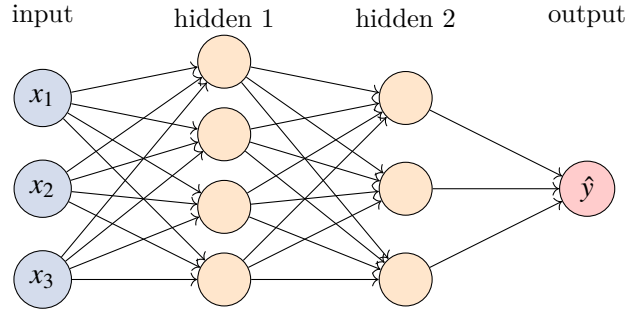
a single perceptron can only learn linearly separable functions (same as a linear SVM). to handle nonlinear problems, u need to stack layers.

4.2 Multi-Layer Perceptron (MLP)

Definition 4.2. an MLP (fully connected feedforward neural network) with L layers:

$$f(x) = \sigma_L(W_L \cdot \sigma_{L-1}(W_{L-1} \cdots \sigma_1(W_1 x + b_1) \cdots + b_{L-1}) + b_L)$$

each layer applies: linear transformation $\rightarrow$ nonlinear activation.



4.3 Activation Functions

sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$ — squashes to $[0, 1]$. used for output probabilities. suffers from vanishing gradients.

tanh: $\sigma(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ — squashes to $[-1, 1]$. zero-centered. also vanishing gradients.

ReLU: $\sigma(z) = \max(0, z)$ — the workhorse of modern deep learning. simple, doesnt saturate for positive inputs. can have “dead neurons” ($z < 0$ always).

softmax (for multi-class output): $\sigma(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ — converts a vector of scores into probabilities.

Intuition. in quantum circuits, the “activation function” is implicit in the measurement process. the born rule ($p = |\langle y | \psi \rangle|^2$) is inherently nonlinear. so a quantum circuit followed by measurement is analogous to a neural network layer with a specific activation function. the key difference: quantum circuits operate with unitary transformations (norm-preserving), while neural networks use arbitrary linear maps + pointwise nonlinearities.

4.4 Backpropagation

Definition 4.3. backpropagation (backprop) is the algorithm for computing gradients of the loss with respect to all parameters in a neural network, using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_l} = \frac{\partial \mathcal{L}}{\partial a_L} \cdot \frac{\partial a_L}{\partial a_{L-1}} \cdots \frac{\partial a_{l+1}}{\partial a_l} \cdot \frac{\partial a_l}{\partial W_l}$$

where a_l is the activation of layer l .

backprop computes all gradients in $O(d)$ time (where d is the number of parameters), which is the same cost as one forward pass. this is why neural networks are trainable — computing gradients is cheap.

Key Point. backprop vs parameter shift rule:

- classical backprop: compute all p gradients with 1 forward + 1 backward pass. cost: $O(1) \times$ forward pass.
- quantum parameter shift: each gradient requires 2 circuit evaluations. cost: $O(p) \times$ circuit evaluation for all p gradients.
- this $O(p)$ overhead is one of the practical bottlenecks of variational quantum algorithms. various tricks (simultaneous perturbation, adjoint methods, etc.) try to reduce it.

4.5 Universal Approximation Theorem

Definition 4.4. the universal approximation theorem (Cybenko 1989, Hornik 1991): a neural network with a single hidden layer and sufficiently many neurons can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy.

this is an existence result — it says the approximation EXISTS but says nothing about how to FIND it (optimization is hard). also, the required number of neurons may be exponentially large.

Key Point. quantum analogue: perez-salinas et al. (2020) showed that data re-uploading (encoding classical data multiple times into a single qubit) makes even a single-qubit circuit a universal function approximator. the quantum version of the universal approximation theorem is: sufficiently deep PQCs with data re-uploading can approximate any function. but again, “sufficiently deep” may be impractically deep, and trainability (barren plateaus) is a concern.

5 Connection Between Kernels and Neural Networks

5.1 Neural Tangent Kernel

Definition 5.1. the neural tangent kernel (NTK) connects neural networks to kernel methods. for a neural network $f_{\theta}(x)$ with parameters θ , the NTK is:

$$K_{\text{NTK}}(x, x') = \left\langle \frac{\partial f_{\theta}}{\partial \theta}, \frac{\partial f_{\theta}}{\partial \theta'} \right\rangle = \sum_i \frac{\partial f_{\theta}(x)}{\partial \theta_i} \frac{\partial f_{\theta}(x')}{\partial \theta_i}$$

in the infinite-width limit, the NTK becomes deterministic and constant during training. the neural network is equivalent to kernel regression with the NTK. this bridges the two paradigms.

Intuition. why does this matter for QML? bcs the same analysis applies to quantum circuits:

- a variational quantum circuit $f_{\theta}(x) = \text{tr}(O \cdot U(\theta, x) \rho U^{\dagger}(\theta, x))$ defines a function of the data and parameters
- the “quantum NTK” is $K_{\text{QNTK}}(x, x') = \sum_i \frac{\partial f}{\partial \theta_i}(x) \frac{\partial f}{\partial \theta_i}(x')$

- schuld (2021) showed that variational quantum models r mathematically equivalent to kernel methods: every variational model implicitly defines a kernel, and the training dynamics can be understood through this kernel
- this unification is one of the deepest results in QML theory

5.2 The Two Paradigms of QML

Key Point. the kernel-NN duality in QML:

quantum kernel methods	variational quantum models
fix the feature map $\phi(x) = U(x) 0\rangle$	train the circuit $U(\theta, x)$
compute kernel $K(x_i, x_j)$ on quantum hardware	optimize θ on classical computer
train SVM/regression classically	end-to-end training
no barren plateaus (no parameter optimization)	barren plateaus possible
inductive bias fixed by circuit design	inductive bias learned during training
well-understood mathematically	harder to analyze theoretically
need quantum computer only for kernel computation	need quantum computer in the training loop

these two approaches r the main paradigms of QML. lectures 9–14 explore both in depth.

6 Deep Learning Architectures (Brief)

6.1 Convolutional Neural Networks (CNNs)

CNNs exploit spatial structure using:

- convolutional layers: apply learnable filters that slide over the input, detecting local patterns
- pooling layers: downsample spatial dimensions
- parameter sharing: the same filter is applied at every position, reducing parameter count

quantum analogue: quantum convolutional neural networks (QCNNs) — circuits with local gates applied in a translational-invariant pattern, followed by measurement-based pooling (tracing out qubits). qcnnns r one of the few QML architectures that provably avoid barren plateaus (Pesah et al. 2021).

6.2 Recurrent Neural Networks (RNNs)

RNNs process sequential data by maintaining a hidden state:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$$

LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) r improved versions that handle long-range dependencies.

quantum analogue: less explored. some proposals for quantum RNNs using sequential quantum circuits, but its an open research area.

6.3 Transformers and Attention

the dominant architecture in modern ML (GPT, BERT, etc.). key mechanism: self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

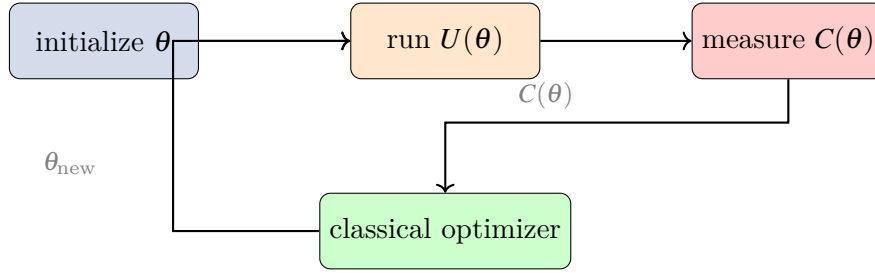
quantum analogues of attention r being explored but r very early stage. the quadratic cost of attention ($O(n^2)$ for sequence length n) has motivated quantum speedup proposals, but practical implementations r far off.

7 The Variational Principle

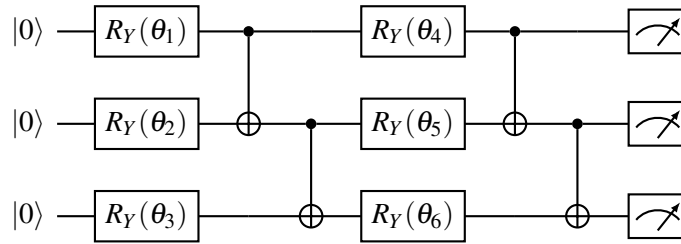
this section bridges classical ML optimization with quantum variational algorithms.

Definition 7.1. the variational principle in quantum computing: given a parameterized quantum circuit $U(\theta)$ and a cost function $C(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$, find the parameters θ^* that minimize C :

$$\theta^* = \arg \min_{\theta} C(\theta) = \arg \min_{\theta} \text{tr}(H \cdot U(\theta) \rho_0 U^\dagger(\theta))$$



a typical variational circuit for VQE looks like this:



the parameters $\theta_1, \dots, \theta_6$ r optimized by the classical optimizer to minimize the cost function. the circuit structure (which gates, which connectivity) is the ansatz — its the “architecture” of the quantum model.

this “quantum-classical loop” is the standard architecture for all variational quantum algorithms:

- VQE (variational quantum eigensolver): $C = \langle H \rangle$ for finding ground state energies
- QAOA: C encodes combinatorial optimization objectives
- variational quantum classifiers: C is a classification loss

- quantum kernel training: C is a kernel alignment objective

Key Point. the variational approach is the dominant paradigm for NISQ quantum computing bcs:

1. circuits can be kept shallow (noise-friendly)
2. the classical optimizer handles the hard optimization work
3. the quantum computer only needs to estimate expectation values (not do complex coherent computation)
4. its flexible — same framework works for chemistry, optimization, and ML

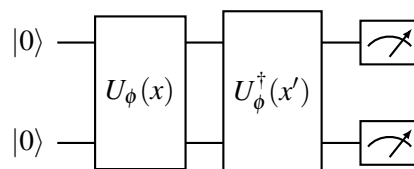
the main challenges: barren plateaus (flat loss landscapes), local minima, noise in gradient estimates, and the $O(p)$ cost of gradient computation via parameter shift.

8 Summary

Key Point. the big connections:

1. SVMs and kernel methods: data is classified by inner products in a feature space. the kernel trick avoids computing the feature map. quantum kernel methods use quantum circuits as feature maps and estimate kernels on quantum hardware.
2. neural networks: parameterized function approximators trained by gradient descent (back-prop). variational quantum circuits r the quantum analogue — parameterized unitaries trained by the parameter shift rule.
3. kernel-NN duality: neural networks in the infinite-width limit r equivalent to kernel methods (NTK). similarly, variational quantum models r equivalent to quantum kernel methods (schuld 2021).
4. the variational principle: the quantum-classical optimization loop. parameterize a quantum circuit, measure a cost function, optimize classically. this is the framework for all NISQ QML algorithms.

a quantum kernel circuit computes similarity between encoded data points:



$P(\text{all zeros}) = |\langle \phi(x') | \phi(x) \rangle|^2 = K_Q(x, x')$. this is the quantum kernel — we'll develop it fully in lectures 12–14.

with the classical ML foundation complete, we're ready to dive into quantum ML properly. next: parameterized quantum circuits — the building blocks of variational QML.